



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

CS7CS4 Machine Learning

Week 3

Shane O'Donnell – 21364002
Dataset ID: # id:8-16--8

Introduction

This weeks investigation entailed investigating the differences between Lasso and Ridge Regression and the effect of varying the regularisation penalty $[C]$ value on the predicted surface for a two feature model. As well as this, using k-fold cross validation and varying the regularisation penalty, the optimal parameters were chosen by measuring the mean squared error for each combination.

Method

In order to begin this investigation, the dataset provided was visualized using a 3D plot to describe how the two features influence the output $[y]$ value. This can be seen in figure 1 below. In order to analyse whether the dataset is situated on a plane or a curve, a contour map was also visualised in figure 2, from the visualisation it can be inferred that the dataset lies on a curve due to its larger target $[y]$ values at the extremes of the feature 1 $[x_1]$ axis and their towards the middle of the axis.

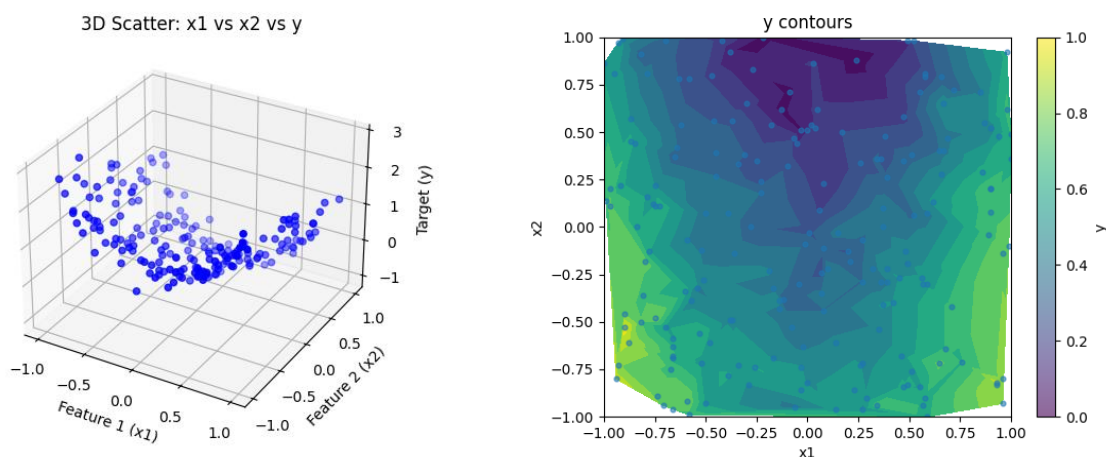


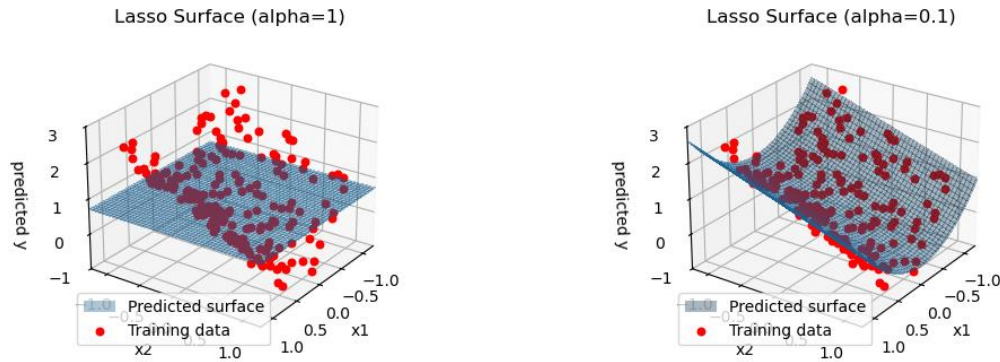
Figure 1: Scatter Plot of the Dataset and contour map of y value in terms of the 2 features

Building on the two features provided by the dataset, extra polynomial features equal to all combinations of powers of the two features up to power 5 were added to the dataset utilising the “PolynomialFeatures” function from sklearn. From here it was necessary to split the data into “train” and “validate” sections to ensure the models to be made are tested on unseen data.

From here it was now possible to begin training the models, the first investigation entailed training a Lasso regression model with varying C values. In order to assess the C values chosen for this investigation, the starting value should offer a model which has all 0 parameters and decreased accordingly. In order to establish the first C value to evaluate, a range of values for C were tested and it was found that the smallest C value with all zero weight coefficients was 1. From here a range of values $[1, 0.1, 0.01, 0.001, 0.0001]$ were utilised to train lasso models using the same train – test split throughout. In order to visualise the characteristics of each model with differing C values, the training data was plotted as a 3D scatter diagram as well as the predicted surface which the model believes is the best fit to capture the trends within the test data. Figure 3 outlines this visualisation for a penalty value of 1 and 0.1.

From the graph and the surface’s formula outlined below, it can be seen that the surface models a flat plain intersecting the curve that is present in the training data and does not take any input features into account. This is a clear sign of underfitting as the model does not capture any of the intricacies present in the training data.

In order to investigate the effect of the penalty value the model was retrained with a penalty of 0.1. This is displayed below in the same format coupled with the model's predicted surface's formula representation. By analysing the graph and equation, the model's predicted surface begins to utilise the feature set at its disposal and is able to begin matching the curve present in the training data.



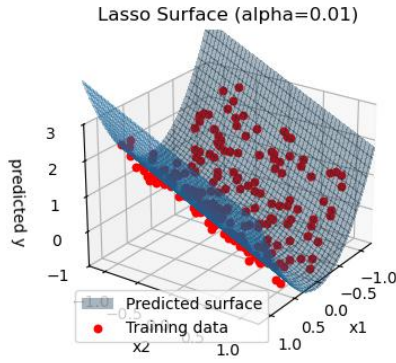
$$\hat{y} = 0.7095$$

$$\hat{y} = 0.3785 + 0.8967 \cdot x_1^2 - 0.6699x_2$$

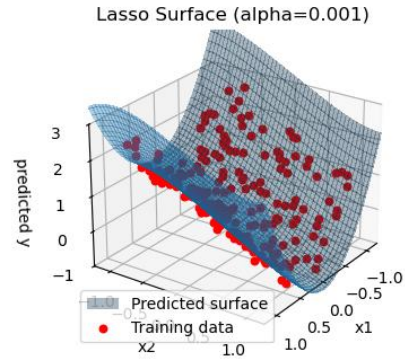
Figure 2: Lasso Regression with a penalty (C) of 1 (left) and 0.1 (right) and the surface formulas

To further this investigation, the penalty value was further decreased to 0.01 and 0.001, by analysing the visualisation of the model with a 0.01 penalty, it can be seen that the model begins to fully capture the intricate behaviours of the training dataset by modelling a curve which is more representative of the contour map which was utilised to analyse whether the training data was a plane or a curve. This model has a more dramatic parabolic shape compared to its 0.1 penalty model counterpart which allows for a more realistic model surface. From the formula associated with the model, the formula consists of one squared x_1 feature $[1.8853x_1^2]$ contributes to the parabolic behaviour of the model while the $[-0.0115x_1]$ offer a slight downward slope on the x_1 plane as x_1 increases and the $[0.9696x_2]$ component create a more dramatic slope across the x_2 axis which creates a dip in the parabolic function which corresponds to a lower predicted y if x_2 is positive instead of negative and x_1 is constant for both cases.

Analysing the 0.001 regularisation model, the lower regularisation value allows for higher order and interaction terms to be assigned a weight the component $[1.9882x_1^2]$ retains the parabolic nature across the x_1 axis while the $[-1.0540x_2]$ component increases the tilt across the x_2 axis. As well as this, the higher order components and interaction terms are assigned smaller weights but create added curvature and intricacies to the predicted surface which may be a sign of overfitting.



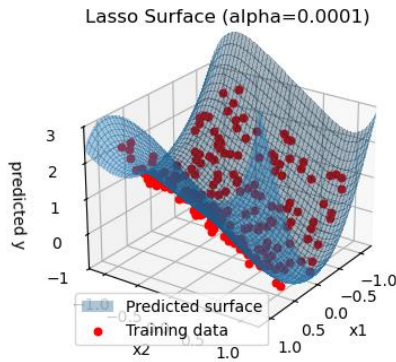
$$\hat{y} = 0.0330 + 1.8853 \cdot x_1^2 - 0.9696 \cdot x_2 - 0.0115 \cdot x_1$$



$$\begin{aligned} \hat{y} = & 0.0178 + 1.9882 \cdot x_1^2 - 1.0540 \cdot x_2 + 0.1191 \cdot x_1^2 x_2^3 + 0.0777 \cdot x_1 x_2^3 - \\ & 0.0590 \cdot x_1 x_2^2 - 0.0588 \cdot x_2^2 + 0.0521 \cdot x_2^5 - 0.0195 \cdot x_1^3 - 0.0018 \cdot x_1^5 + \\ & 0.0973 \cdot x_1^3 - 0.0824 \cdot x_1 x_2^2 - 0.0668 \cdot x_1^4 x_2 + 0.0496 \cdot x_1^3 x_2^2 + \\ & 0.0422 \cdot x_1^2 x_2^2 - 0.0143 \cdot x_1 x_2^4 \end{aligned}$$

Figure 3: Lasso Regression with a penalty (C) of 0.01 (left) and 0.001 (right) and their associated surface formulas

The final model trained with Lasso Regression model was trained with a regularisation strength of 0.0001. This can be seen in figure 5 as well as the formula corresponding the surface. From the surface visualised, it can be inferred that the model is overfitting to the training data as its complexity has grown immensely and captures a significant amount of intricacies in the training data compared to its predecessor models.



$$\begin{aligned} \hat{y} = & 0.0232 + 2.1228 \cdot x_1^2 - 0.8545 \cdot x_2^3 - 0.8330 \cdot x_2 + 0.6889 \cdot x_1^2 x_2^3 + 0.6606 \cdot x_2^5 - \\ & 0.4355 \cdot x_1 x_2^2 + 0.2943 \cdot x_1 x_2^4 - 0.2706 \cdot x_2^2 + 0.2600 \cdot x_1 x_2^3 + 0.2179 \cdot x_2^4 - \\ & 0.1957 \cdot x_1^2 x_2 - 0.1618 \cdot x_1 x_2 + 0.1591 \cdot x_1^3 x_2^2 - 0.1372 \cdot x_1^4 x_2 - 0.1154 \cdot x_1^4 - \\ & 0.0943 \cdot x_1^5 + 0.0815 \cdot x_1^3 x_2 + 0.0557 \cdot x_1 + 0.0324 \cdot x_1^3 \end{aligned}$$

Figure 4: Lasso Regression with a penalty (C) of 0.0001

During the analysis of this investigation, the use of describing the surfaces as “underfitting” and “overfitting” was used. These terms describe the extent that the trained model captures the intricacies and complex behaviour of the training data. An example of underfitting can be seen in the plot which displays the model trained with a regularisation strength of 1; the model fails to represent the underlying structure of the data; this is inferred by its flat surface prediction and the fact that from our contour map it can be seen the training data lies on a curve. By applying a regularisation strength

that is too strong for lasso regression the polynomial components are forced to 0 which causes the surface to lack the ability to model the curve.

The term overfitting refers to a model becoming too complex and has the ability to rely too heavily on the training data, although this will cause the model to perform very well on the training set, when tested with the test set, it will perform worse due to being tailored to the training dataset and lacking versatility on unseen data. An example of this can be seen with our lasso model with a regularisation strength of 0.0001. To visualise how to detect underfitting and overfitting, the mean square error was calculated for each regularisation

strengths training and test model. 5 fold cross validation was used to test and train on different segments of the dataset and the average mse was logged for each regularisation strength. This can be seen visualised in figure 6. Underfitting can be identified as the models with regularisation strength 10^0 and higher as their mse for training is very high and so is the test model mse. Overfitting can be identified in the two lowest regularisation strengths as the training model has an extremely low mse but the training model fails to reach the same performance. The optimal model can be identified as the model with 10^{-2} as its training and test models perform almost equally as well. From analysing the standard deviation of the MSE value computer for different folds at the same regularisation penalty it can be inferred that the test set's performance can vary greatly when the model is overfit or underfit whilst the model with the idea c value [10^{-2}] is more stable and performs very similar across different folds.

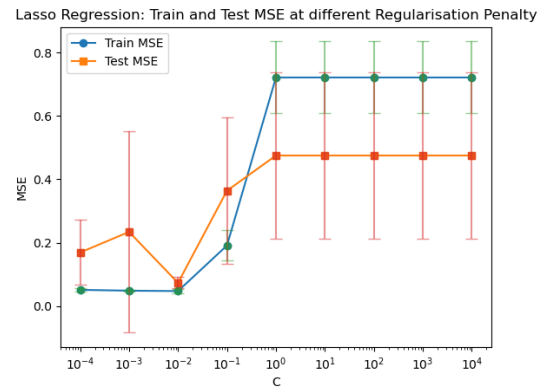
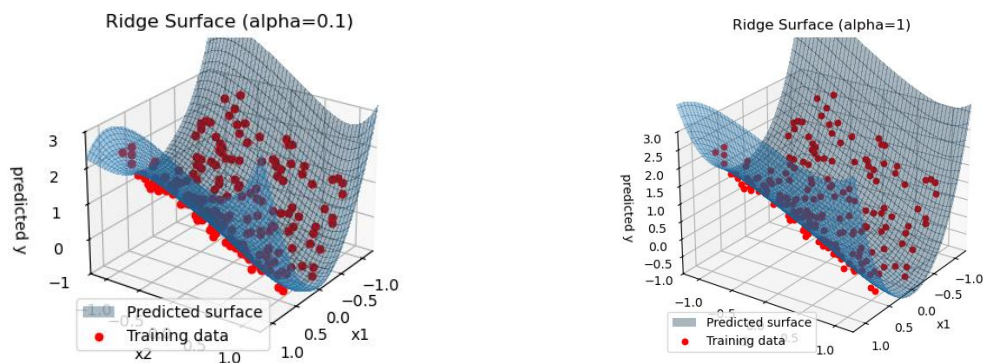


Figure 5: Lasso Regression MSE with 5 fold cross validation for train and test sets

To further highlight the effect of regularisation and its effects on the underfitting and overfitting of a model. The experiments undertaken for the Lasso Regression models were repeated but using Ridge Regression instead. Figure 7 outlines the model's predicted surfaces for Ridge Regression Models with a regularisation strength of 0.1 and 1. An initial indication of the difference between Ridge and Lasso regression is the fact that the formulas shown below do not set any features to 0; this is because Ridge Regression uses L2 regularisation which adds the sum of squared coefficients which shrinks the coefficients gradually but does not set any to 0. Both models share similar behaviour with a strongly weighted squared x_1 feature which creates the parabolic nature across the x_1 axis and a strongly weighted negative x_2 term which creates the tilt effect across the x_2 axis. Although from the graph and formulas it can be seen that the 0.1 regularisation model have strongly weighted x_2 polynomials which creates more twists along the x_2 axis. In comparison the 1 regularisation model less strong high order x_2 features which makes the bowl more symmetric.



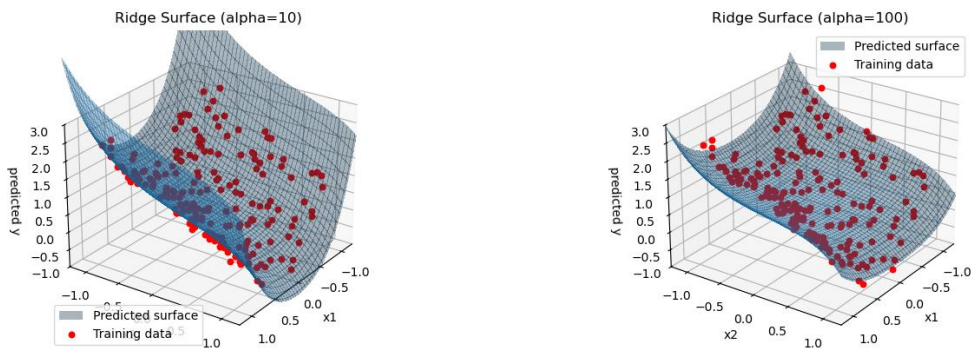
$$\begin{aligned}\hat{y} = & 0.0501 + 1.9798 \cdot x_1^2 - 0.9287 \cdot x_2 \\ & + 0.5315 \cdot x_1^2 x_2^3 - 0.4370 \cdot x_2^3 + 0.3286 \cdot x_2^5 \\ & - 0.2404 \cdot x_2^2 + 0.1793 \cdot x_1 x_2^3 + 0.1715 \cdot x_2^4 \\ & + 0.1471 \cdot x_1^3 x_2 - 0.1462 \cdot x_1^5 - 0.1252 \cdot x_1^2 x_2 \\ & - 0.1241 \cdot x_1 x_2 + 0.0131 \cdot x_1^4 - 0.0000 \cdot x_1\end{aligned}$$

$$\begin{aligned}\hat{y} = & 0.1152 + 1.4258 \cdot x_1^2 - 0.9389 \cdot x_2 \\ & + 0.4986 \cdot x_1^4 - 0.1958 \cdot x_2^3 + 0.1919 \cdot x_1^2 x_2^3 + 0.1557 \cdot x_1^2 x_2^2 \\ & - 0.1512 \cdot x_2^2 + 0.1274 \cdot x_2^5 + 0.0923 \cdot x_1^3 x_2 \\ & + 0.0763 \cdot x_1 x_2^3 - 0.0613 \cdot x_1^2 x_2 + 0.0597 \cdot x_2^4 \\ & - 0.0395 \cdot x_1^5 - 0.0359 \cdot x_1^3 - 0.0320 \cdot x_1^3 x_2^2 - 0.0291 \cdot x_1 x_2^4 \\ & - 0.0208 \cdot x_1 x_2^2 + 0.0155 \cdot x_1^4 x_2 - 0.0149 \cdot x_1 x_2 + 0.0138 \cdot x_1\end{aligned}$$

Figure 6: Ridge Regression with a penalty (C) of 0.1 (left) and 1 (right) and the surface formulas

The next models visualised and analysed were at regularisation strengths 10 and 100. This can be seen figure 8 which displays the predicted surfaces, and the formula associated with them. Compared to the first two ridge models, the overall shape and behaviour is similar but have assigned lower weights to the squared x_1 feature. By lowering this x_1 squared feature the parabolic bowl becomes less tight which can be seen in the 100-regularisation model as the bowl begins to flatten.

[maybe add more about the components of function]

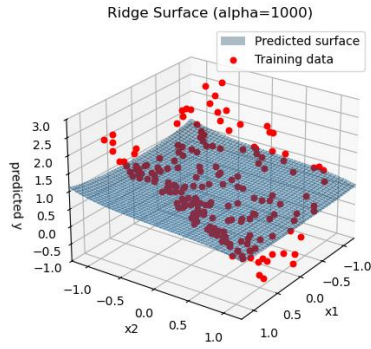


$$\begin{aligned}\hat{y} = & 0.2540 + 0.8455 \cdot x_1^2 - 0.6667 \cdot x_2 + 0.6520 \cdot x_1^4 \\ & - 0.2287 \cdot x_2^3 + 0.2261 \cdot x_1^2 x_2^2 - 0.0855 \cdot x_1^2 x_2 \\ & - 0.0715 \cdot x_2^5 + 0.0543 \cdot x_1 x_2 - 0.0468 \cdot x_2^2 \\ & - 0.0344 \cdot x_1^3 + 0.0325 \cdot x_1^3 x_2 - 0.0293 \cdot x_1^3 x_2^2 \\ & - 0.0238 \cdot x_1 - 0.0228 \cdot x_1 x_2^2 - 0.0205 \cdot x_1 x_2^4 - 0.0110 \cdot x_1^5 \\ & - 0.0092 \cdot x_2^4 + 0.0038 \cdot x_1^2 x_2^3 + 0.0026 \cdot x_1^4 x_2 + 0.0021 \cdot x_1 x_2^3\end{aligned}$$

$$\begin{aligned}\hat{y} = & 0.5516 - 0.2862 \cdot x_2 + 0.2558 \cdot x_1^2 + 0.2199 \cdot x_1^4 \\ & - 0.1593 \cdot x_2^3 - 0.1052 \cdot x_2^5 + 0.0848 \cdot x_1^2 x_2^2 \\ & - 0.0740 \cdot x_1^2 x_2 - 0.0439 \cdot x_1^2 x_2^3 - 0.0315 \cdot x_1^4 x_2 \\ & - 0.0295 \cdot x_1 + 0.0237 \cdot x_1 x_2 + 0.0139 \cdot x_2^4 \\ & - 0.0136 \cdot x_1^3 + 0.0132 \cdot x_2^2 + 0.0082 \cdot x_1^3 x_2 - 0.0077 \cdot x_1 x_2^2 \\ & - 0.0042 \cdot x_1^3 x_2^2 - 0.0035 \cdot x_1 x_2^4 + 0.0021 \cdot x_1 x_2^3 - 0.0004 \cdot x_1^5\end{aligned}$$

Figure 7: Ridge Regression with a penalty (C) of 10 (left) and 100 (right) and the surface formulas

The final model visualised was with a regularisation strength of 1000. Similar to the Lasso surface with a regularisation of 1, this ridge model begins to predict a surface which is flat and does not capture the curved nature of the test data. Unlike the Lasso model, due to L2 regularisation not setting components to 0, the ridge model assigns very small weights to the coefficients and utilises a large offset [0.6975] to create a surface with minimal curvature and prediction values that diverge minimally from the offset value.



$$\hat{y} = 0.6975 - 0.0526 \cdot x_2 + 0.0318 \cdot x_1^2 - 0.0317 \cdot x_2^3 + 0.0274 \cdot x_1^4 - 0.0225 \cdot x_2^5 - 0.0161 \cdot x_1^2 x_2 + 0.0111 \cdot x_1^2 x_2^2 - 0.0103 \cdot x_1^2 x_2^3 - 0.0083 \cdot x_1^4 x_2 - 0.0051 \cdot x_1 + 0.0039 \cdot x_2^2 + 0.0038 \cdot x_1 x_2 + 0.0035 \cdot x_2^4 - 0.0022 \cdot x_1^3 + 0.0014 \cdot x_1^3 x_2 + 0.0009 \cdot x_1 x_2^3 - 0.0005 \cdot x_1 x_2^2 + 0.0003 \cdot x_1 x_2^4 - 0.0002 \cdot x_1^5 + 0.0000 \cdot x_1^3 x_2^2$$

Figure 8: Ridge Regression with a penalty (C) of 1000 and the surface formulas

To assess the presence of underfitting or overfitting, a graph visualising the mean squared error of the train and test models and the corresponding regularisation amount used to create these models was created. From the graph apparent in figure 9 underfitting is apparent at greater regularisation values [10^3 and 10^4] while overfitting is apparent at the lower regularisation strengths [10^{-4} to 10^{-2}]. The regularisation strength which offers train and test models that perform very similarly is at 10^{-1} and 10^0 . The standard deviation of the MSE scores for a given c value with different folds were also displayed, it can be seen at a regularisation penalty of 10^{-4} the overfitting of the model causes the MSE varies greatly at different test folds. While when the regularisation penalty is at an ideal value [10^0] the standard deviation is negligible.

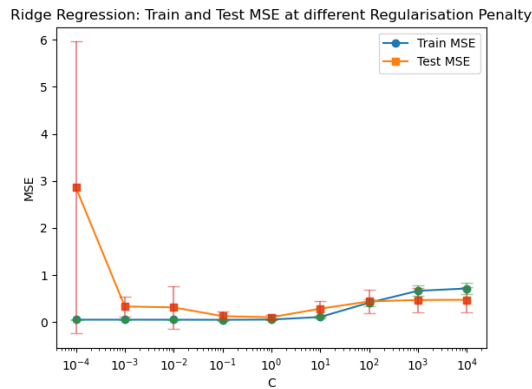


Figure 9: Ridge Regression MSE with 5 fold cross validation for train and test sets

Code:

```
import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

import numpy as np

import pandas as pd

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import Lasso

from sklearn.linear_model import Ridge

from sklearn.model_selection import cross_val_score

from sklearn.model_selection import train_test_split


#load in data

df = pd.read_csv('week3.csv', header=None)

print(df.head())

X1 = df.iloc[:,0].values

X2 = df.iloc[:,1].values

X = np.column_stack([X1, X2])

y = df.iloc[:,2].values

print(X.shape, y.shape)


#plot the dataset on a 3D scatter plot - first feature on x, second on y and target on z

fig = plt.figure()

ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:,0], X[:,1], y, c='b', marker='o')


ax.set_xlabel("Feature 1 (x1)")

ax.set_ylabel("Feature 2 (x2)")

ax.set_zlabel("Target (y)")

ax.set_title("3D Scatter: x1 vs x2 vs y")


plt.show()


#contour map of y against x1 and x2 to assess if plane or curved surface

plt.figure()
```



```
plt.tricontourf(X[:,0],X[:,1],y,levels=15)
plt.scatter(X[:,0],X[:,1],s=12,alpha=0.6)
plt.xlabel("x1");
plt.ylabel("x2");
plt.title("y contours");
plt.colorbar(label="y")
plt.show()
```

```
#in addition to the two features in data file add extra polynomial features equal to all combinations of powers
#of x1 and x2 up to the 5th power
#use
poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)
print(X_poly.shape) #check new shape of the features
```

```
#split data into train and split
#use 80% of the data for training and 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
```

```
#train lasso regression model on the data with polynomial features with changing c values
#c value should start small enough that all trained coeffs are zero then increase accordingly
#c_val = [0.001, 0.01, 0.1, 1, 10, 100] #TO:DO: change these values to ensure it starts with all coeffs zero
c_val = [10000,1000,100,10,1,0.1,0.01,0.001,0.0001] #changed these values to ensure it starts with all coeffs
zero
#for lasso the bigger alpha (c) the more coeffs are zero
```

```
#store models here for use later
models = []
```

```
for c in c_val: #iterate through the c value array
    lasso_model = Lasso(alpha=c, max_iter=10000) #create lasso model with c value
    lasso_model.fit(X_train, y_train) #fit the model to the data
    coeffs = lasso_model.coef_ #get the coeffs
```

```

print(f'Lasso Coefficients C={c}: {coeffs}\n')

#store model currently used
models.append((c, lasso_model))

#=====subject to change=====

extension = 0.25 #extends beyond data a bit as per assignment brief

x1_min, x1_max = X[:,0].min()-extension, X[:,0].max()+extension #set the min and max for the 2 features
and broaden range with the extension value

x2_min, x2_max = X[:,1].min()-extension, X[:,1].max()+extension #change the extension value to get the best
view

#create grid - use the range defined above to define the limits of the grid
grid_x1 = np.linspace(x1_min, x1_max, 150)
grid_x2 = np.linspace(x2_min, x2_max, 150)

#for loop taken from the assignment brief
Xtest = []

#change these around to match surface to x1 and x2 axis
for i in grid_x1:
    for j in grid_x2:
        Xtest.append([i, j])
Xtest = np.array(Xtest)

#expand with polynomial features - this matches the polynomial features used in training
Xtest_poly = poly.transform(Xtest)

#meshgrid creates 2d array of the x1 and x2 values for plotting
X1grid, X2grid = np.meshgrid(grid_x1, grid_x2)

zmin, zmax = -1.0, 3.0 #match raw data y output range to lock z axis on plots

for c, lasso_model in models:
    Z = lasso_model.predict(Xtest_poly)#predict using the transformed test data and reshape to match the grid
    shape

```

```

gridx1_len, gridx2_len = len(grid_x1), len(grid_x2)
Z = Z.reshape(gridx2_len, gridx1_len).T
#Z = lasso_model.predict(Xtest_poly).reshape(X1grid.shape) #reshape to match the grid shape
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
#ax.set_proj_type('ortho') # less perspective distortion
#plot surface and training points
surf = ax.plot_surface(X1grid, X2grid, Z, alpha=0.35, linewidth=0, antialiased=False, label='Predicted
surface')

ax.scatter(X[:,0], X[:,1], y, s=22, c='r', depthshade=False, label='Training data')

#lock axes to the tight box to avoid the problem of massive autoscaling
ax.set_xlim(grid_x1.min(), grid_x1.max())
ax.set_ylim(grid_x2.min(), grid_x2.max())
ax.set_zlim(zmin, zmax)

ax.view_init(elev=25, azim=35) #change to suit the viewing angle
ax.set_xlabel("x1");
ax.set_ylabel("x2");
ax.set_zlabel("predicted y")
ax.set_title(f'Lasso surface (alpha={c})')
ax.legend()
plt.show()

#
#(d) - explain this part in report

#(e) - redo but with ridge regressing instead of lasso regression
#add extra polynomials
#train ridge models with different c values
#gnerate predictions and plot surfaces

#c values for ridge regression - smaller values mean more regularisation
#reuse c_val from before but iterate backwards

```

```

#use x_poly from before
c_val_ridge = c_val[::-1] #reverse the order of c values
models_ridge = [] #store ridge models here
for c in c_val_ridge:
    ridge_model = Ridge(alpha=c, max_iter=10000) #ridge model with changing c value
    ridge_model.fit(X_poly, y) #fit the model to the data
    coeffs = ridge_model.coef_ #get the coeffs
    print(f'Ridge Coefficients (c={c}): {coeffs}\n')
    models_ridge.append((c, ridge_model)) #store the model

#get predictions and plot the surfaces for each model

#already have x1_min, x1_max, x2_min, x2_max from before
#already have grid_x1, grid_x2 from before
#can maybe get ride of this as well
Xtest = []
grid=np.linspace(-5,5)
for i in grid:
    for j in grid:
        Xtest.append([i,j])
Xtest = np.array(Xtest)
#expand with polynomial transformer - already done for lasso so reuse

for c, ridge_model in models_ridge:
    ridge_Z = ridge_model.predict(Xtest_poly) #predict using the transformed test data and reshape to match the
    grid shape
    gridx1_len, gridx2_len = len(grid_x1), len(grid_x2)
    ridge_Z = ridge_Z.reshape(gridx2_len, gridx1_len).T
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')

    surface_ridge = ax.plot_surface(X1grid, X2grid, ridge_Z, alpha=0.35, linewidth=0, antialiased=False,
    label='Predicted surface')

    ax.scatter(X[:,0], X[:,1], y, s=22, c='r', depthshade=False, label='Training data')

```

```

#lock axes to avoid massive autoscaling
ax.set_xlim(grid_x1.min(), grid_x1.max())
ax.set_ylim(grid_x2.min(), grid_x2.max())
ax.set_zlim(zmin, zmax)

ax.view_init(elev=25, azim=35)
ax.set_xlabel("x1"); ax.set_ylabel("x2"); ax.set_zlabel("predicted y")
ax.set_title(f'Ridge Surface (alpha={c})')
ax.legend()
plt.show()

```

```

#use 5 fold cross validation to plot mean and standard dev of the prediction error vs C
#use errorbar funtion
mean_errors_lasso = []
std_errors_lasso = []
mean_errors_ridge = []
std_errors_ridge = []
for c in c_val:
    lasso_model = Lasso(alpha=c, max_iter=10000)
    #cross_val_score uses negative MSE
    neg_mse_scores = cross_val_score(lasso_model, X_poly, y, cv=5, scoring='neg_mean_squared_error')
    #convert to pos
    mse_scores = -neg_mse_scores
    print(mse_scores)
    ##append to array
    mean_errors_lasso.append(np.mean(mse_scores))
    std_errors_lasso.append(np.std(mse_scores))

#do the same for ridge
for c in c_val_ridge:
    ridge_model = Ridge(alpha=c, max_iter=10000)
    #cross_val_score uses negative MSE
    neg_mse_scores = cross_val_score(ridge_model, X_poly, y, cv=5, scoring='neg_mean_squared_error')
    #convert to pos

```

```

mse_scores = -neg_mse_scores

print(mse_scores)

##append to array

mean_errors_lasso.append(np.mean(mse_scores))

std_errors_lasso.append(np.std(mse_scores))


#make the plots

plt.figure()

plt.errorbar(c_val, mean_errors_lasso, yerr=std_errors_lasso, fmt='-o', label='Lasso', capsize=5)

plt.errorbar(c_val, mean_errors_ridge, yerr=std_errors_ridge, fmt='-s', label='Ridge', capsize=5)

plt.xscale('log')

plt.xlabel('C')

plt.ylabel('MSE')


plt.title('Mean Squared Error vs C for Lasso and Ridge Regression')

plt.legend()

plt.show()


#lassos final zero coeffs is 1

#ridge never zeros out but coeffes get very small visibly at 1000


# =====make plots for mse of train and test data against c values for both lasso and ridge=====

train_mse_lasso = []

test_mse_lasso = []

train_mse_ridge = []

test_mse_ridge = []

train_mse_lasso_std = []

test_mse_lasso_std = []

train_mse_ridge_std = []

test_mse_ridge_std = []

#maybe reuse c_val and c_val_ridge and see if i saved the models in an array with their c values

for c, lasso_model in models:

    #use cross_val_score to get mse for train data

```

```

neg_mse_scores = cross_val_score(lasso_model, X_train, y_train, cv=5, scoring='neg_mean_squared_error')

mse_scores = -neg_mse_scores

train_mse_lasso.append(np.mean(mse_scores))

##add in std dev

train_mse_lasso_std.append(np.std(mse_scores))

print(train_mse_lasso_std)

#get mse vals for test data - make another array above to hold them so i can do it in the same format

#use crossval_score to get mse for test data

neg_mse_scores_test = cross_val_score(lasso_model, X_test, y_test, cv=5,
scoring='neg_mean_squared_error')

mse_scores_test = -neg_mse_scores_test

test_mse_lasso.append(np.mean(mse_scores_test))

##add in std dev

test_mse_lasso_std.append(np.std(mse_scores_test))


#copy for ridge

for c, ridge_model in models_ridge:

    #use cross_val_score to get mse for train data

    neg_mse_scores = cross_val_score(ridge_model, X_train, y_train, cv=5, scoring='neg_mean_squared_error')

    mse_scores = -neg_mse_scores

    train_mse_ridge.append(np.mean(mse_scores))

    #add in std dev of mse so i can plot error bars on the graph

    train_mse_ridge_std.append(np.std(mse_scores))

    #get mse vals for test data - make another array above to hold them so i can do it in the same format

    #use crossval_score to get mse for test data

    neg_mse_scores_test = cross_val_score(ridge_model, X_test, y_test, cv=5,
scoring='neg_mean_squared_error')

    mse_scores_test = -neg_mse_scores_test

    test_mse_ridge.append(np.mean(mse_scores_test))

    ##add in std dev

    test_mse_ridge_std.append(np.std(mse_scores_test))


#debug std dev arrays

#10 c values are used so the std dev arrays should also have 10 values

print(len(c_val))

```



```

print(f'Lasso Train Std Dev: {train_mse_lasso_std}')
print(f'Lasso Test Std Dev: {test_mse_lasso_std}')
print(f'Ridge Train Std Dev: {train_mse_ridge_std}')
print(f'Ridge Test Std Dev: {test_mse_ridge_std}')

#lasso plot
plt.figure()
#plot mean mse scores
plt.plot(c_val, train_mse_lasso, '-o', label='Train MSE', markersize=6)
plt.plot(c_val, test_mse_lasso, '-s', label='Test MSE', markersize=6)
#plot std dev as error bars
plt.errorbar(c_val, train_mse_lasso, yerr=train_mse_lasso_std, fmt='o', capsize=5, alpha=0.5)
plt.errorbar(c_val, test_mse_lasso, yerr=test_mse_lasso_std, fmt='s', capsize=5, alpha=0.5)
plt.xscale('log')
plt.xlabel('C')
plt.ylabel('MSE')
plt.title('Lasso Regression: Train and Test MSE at different Regularisation Penalty')
plt.legend()
plt.show()

plt.figure()
plt.plot(c_val_ridge, train_mse_ridge, '-o', label='Train MSE', markersize=6)
plt.plot(c_val_ridge, test_mse_ridge, '-s', label='Test MSE', markersize=6)
plt.errorbar(c_val_ridge, train_mse_ridge, yerr=train_mse_ridge_std, fmt='o', capsize=5, alpha=0.5)
plt.errorbar(c_val_ridge, test_mse_ridge, yerr=test_mse_ridge_std, fmt='s', capsize=5, alpha=0.5)
plt.xscale('log')
plt.xlabel('C')
plt.ylabel('MSE')
plt.title('Ridge Regression: Train and Test MSE at different Regularisation Penalty')
plt.legend()
plt.show()

```